智银通 V3.1 技术文档

可信自主银行智能体 | 正式参赛交付版

本文面向比赛评审与技术复核，说明当前系统如何把自然语言目标转化为受控任务，并将资金操作限制在独立模拟账本。完整安全验收 90 项通过；生产身份、真实 MFA 与银行连接不在已验证范围。

版本基准：V3.1 安全修订版 | 资料核验日期：2026 年 11 月 1 日

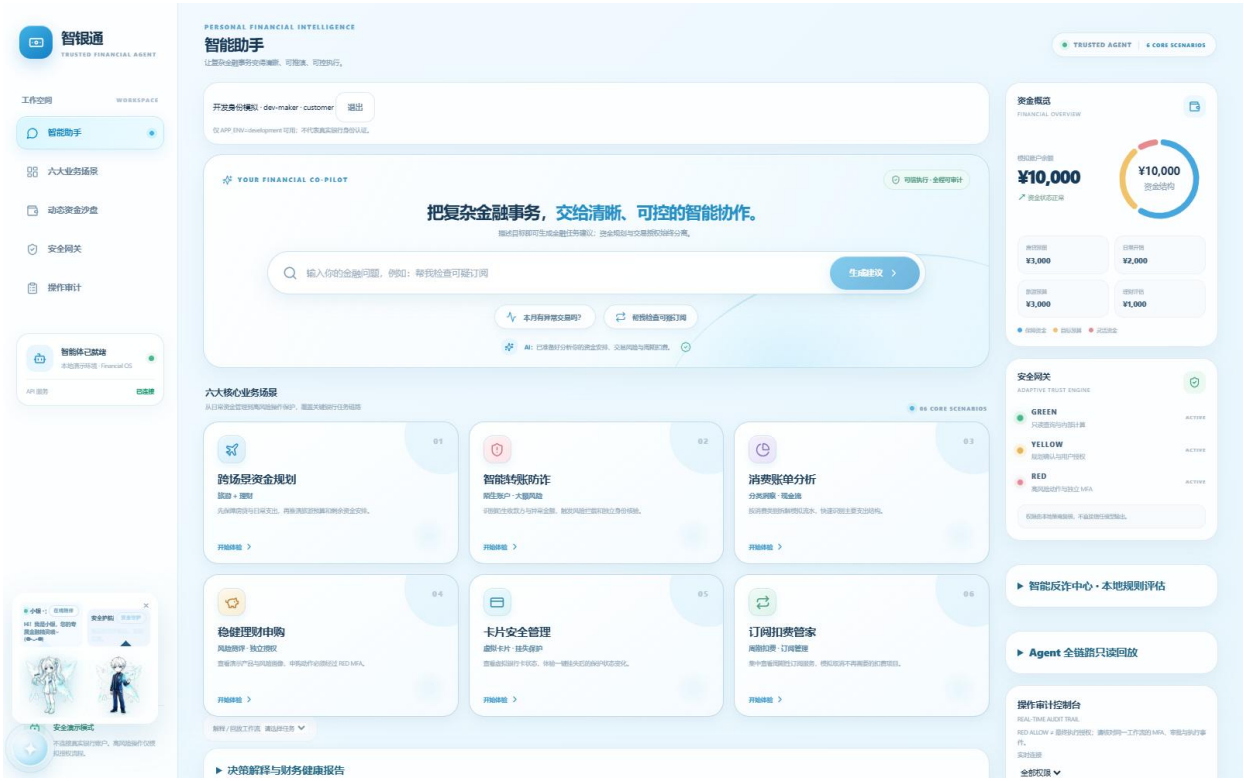


图 1　当前运行界面与双数字精灵。采集于独立临时模拟环境。

阅读导航

1—5 项目与产品架构；6—8 智能体与 Financial DAG；9—19 安全控制与实时通信；20—22 数据、部署与测试；23—25 价值、路线图与边界。源码路径及行号以本次冻结文件为准。

01 项目概述

智银通 ZHIYINTONG V3.1 是面向个人金融任务的可信自主银行智能体演示系统。系统把自然语言目标转换为结构化任务、权限判断、资金约束预演和可追溯的模拟执行。用户能够查看规划依据，而资金写操作始终由服务端执行网关决定是否放行。

本交付版以当前冻结源码和 2026 年 11 月 1 日验收证据为基础。完整测试批次 90 项通过、0 项失败、0 项跳过。该结论适用于本地模拟系统；系统未连接真实银行，开发身份及 MFA 均为明确隔离的模拟实现。V3.1 是安全修订版本，兼容 API 的版本标识仍为 3.0.0。

实现与证据 FINAL_SECURITY_ACCEPTANCE_V3_1.md；backend/test-results/v31-security/full/v31-acceptance.xml

02 金融科技行业背景与业务痛点

个人金融任务通常同时涉及账户、支付、预算、产品适配和风险控制。例如“下个月旅游，保证房贷，剩余理财”包含不同依赖和不同授权强度，单一问答无法表达这些执行边界。自然语言模型还可能产生不完整参数、错误权限或不符合资金约束的计划。

本项目针对上述工程问题设计可检查的中间表示、受控工具目录和服务端授权链。目标是降低任务理解与操作衔接的复杂度，同时保留人的最终控制。这里描述的是设计动机，不是经用户调研验证的规模、效率提升或商业转化数据。

实现与证据 backend/main.py；backend/agent_runtime.py:180、376

03 产品定位与目标用户

产品定位为可信自主银行智能体：通过对话表达目标，通过 Financial DAG 展示任务结构，通过动态卡片承载确认和交互，通过审计记录解释后端实际决定。目标用户包括需要预算规划与风险提示的个人用户，以及希望验证智能体安全编排的金融科技开发团队。

当前形态适合比赛演示、教学及银行创新团队的概念验证。目标客户与落地渠道属于待验证假设，不宣称已服务银行、拥有真实用户或获得机构合作。未来进入银行环境之前，需要完成机构身份接入、合规评估和正式连接器审查。

实现与证据 frontend/pages/index.js；backend/services/auth_service.py:47

04 产品总体架构

前端使用 Next.js 与 React，通过 FastAPI 接口提交意图并接收 SSE 事件。主控编排器选择领域 Agent，生成 Financial IR、DAG 与受控业务卡片。TrustEngine 对动作、身份委托链和权限要求进行复核，模型不拥有账户写权限。

独立金融动作进入 ExecutionGateway。网关在 SQLite 事务内协调授权快照、幂等记录、沙箱复验、Maker/MFA、必要的 Checker 审批、模拟账本与审计。Agent、MCP 和浏览器均不能直接写账本。架构图中的“执行”特指独立模拟账本提交，不代表银行清算。

实现与证据 backend/services/execution_gateway.py:13、126；backend/services/database.py



图 2 总体架构。箭头为主要逻辑流，执行仅连接模拟账本。

05 前端交互与可视化系统

前端保留蓝白视觉、双数字精灵、六大业务入口、Financial DAG 和声明式业务卡片。UIBusinessCard 的 Pydantic 校验限定卡片类型及数据结构，React 从受控组件目录渲染，避免直接执行模型生成的任意界面代码。

精灵通过工作流事件呈现分析、风险提示与等待状态；AuditConsole 读取后端审计，不能把前端 local state 当作执行凭证。截图来自当前页面连接独立临时后端的真实交互；未改动核心代码，也未使用虚构产品界面。DAG 动画表示校验和编排进度，不等于资金已提交。

实现与证据 frontend/pages/index.js；frontend/components/AuditConsole.jsx；backend/agent_runtime.py:132

06 Multi-Agent 多智能体协同架构

MasterOrchestrator 通过显式场景键或关键词分类选择领域技能。六个主要领域分别覆盖旅游资金规划、转账风险、账单分析、理财适配、卡片保护和订阅管理；内部另有异常分析及通用回退。DomainAgent 产生经过 Pydantic 校验的节点、边和业务卡片。

当前多智能体协同属于主控路由与确定性领域技能组合，并非多个独立大模型自由协商。可选在线模型用于语义提取，离线或结构校验失败时回退本地规则。此设计把可变的语言理解与固定的金融约束分开，便于复现和测试。

实现与证据　backend/agent_runtime.py:376、684、711；backend/main.py



图 3　主控与六类领域 Agent。内部还保留异常及通用回退。

07 金融意图理解与场景路由

输入先经 SecurityGuardrail 检查，再提取目标、金额、场景及约束。后端以类型化数据形成 Financial IR，领域 Agent 构建场景任务。模型不可指定最终角色、提高权限或绕过受保护预留，服务端保留参数与策略的最终解释权。

旅游输入的演示结果使用余额 10,000 元、房贷 3,000 元、日常支出 2,000 元和最低流动性 1,000 元，得到安全可分配额 4,000 元，再分配旅游预算 3,000 元及理财评估余额 1,000 元。结果是固定约束下的模拟规划，不是收益建议或真实授信。

实现与证据　backend/agent_runtime.py:258、389；backend/main.py；真实截图 assets/travel-budget.png

08 Financial DAG 任务编排机制

节点描述任务标识、动作、输入输出及 GREEN/YELLOW/RED 等级，依赖边定义先后关系。沙箱检查节点唯一性、依赖存在性和无环性。旅游场景的真实运行结果为 20 个节点、27 条依赖边，展示数据采集、约束计算、预算确认、独立授权与模拟回执的关系。

前端细化 DAG 用于解释计划；实际执行以服务端工作流和授权快照为准。规划确认只确认资金分配，不自动授权理财申购。图中 RED 节点即使显示结构校验完成，也不能替代交易的 Maker/MFA/Checker 状态检查。

实现与证据 backend/services/sandbox_service.py:32；backend/agent_runtime.py:558；frontend/pages/index.js:747



图 4 旅游 Financial DAG 简化语义视图；实际为 20 节点、27 条边。

09 TrustEngine 五平面安全架构

为说明现有实现，安全能力归纳为五个平面：输入与内容、身份与会话、委托链、工具与权限、执行与数据。输入平面对应提示检查和脱敏；身份平面使用服务端会话；委托链要求 human、agent、connector 的受控关系；工具平面采用动作目录和权限等级；执行平面负责快照、事务、账本及审计。

五平面是对当前代码控制点的架构映射，不代表已经实现完整零信任网络、硬件密钥管理或银行生产合规体系。TrustEngine 的局部 ALLOW 只是某次策略判断，执行网关仍须检查全部授权及资金条件。

实现与证据 backend/agent_runtime.py:180、258、452；backend/services/auth_service.py；backend/services/execution_gateway.py:126



图 5 五平面安全能力与现有控制点的映射。

10 GREEN YELLOW RED 权限控制机制

GREEN 适用于只读查询与内部计算；YELLOW 要求明确用户确认；RED 涉及高风险模拟动作，必须完成 Maker 确认及独立模拟 MFA。权限由服务端动作目录决定，不接受客户端自行声明升级。

当交易金额严格大于 50,000 元时，RED 流程额外进入 HITL 与独立 supervisor 审批。50,000 元本身不触发这一金额规则。默认账户受保护预留后仅有 4,000 元可用，因此默认账户的 8,800 元或 50,001 元动作会先被沙箱拒绝；大额授权测试使用专用合成高余额账户。

实现与证据 backend/agent_runtime.py:180；backend/services/execution_gateway.py:52、126；backend/tests/test_policy_binding_v31.py

11 Sandbox 隔离执行机制

SandboxService 使用独立模拟账户与模拟分录，金额采用整数分，避免浮点误差。校验范围包括金额格式、收款对象白名单、动作权限、DAG 结构、风险标志、余额及保护性预留。执行时再次读取当前账户条件，不能复用过时余额判断。

初始沙箱结果与 sandbox_run_id、结果哈希一起写入不可变授权快照。最终提交使用带余额条件的更新及唯一工作流分录，保证余额不能侵占受保护资金。数字孪生的 30 天预测属于只读情景计算，独立于资金执行沙箱。

实现与证据 backend/services/sandbox_service.py:15、32、67；backend/services/database.py

12 Idempotency 防重放机制

客户端以 UUID 幂等键表达同一意图，服务端按身份及操作作用域记录幂等键和请求摘要。重复键配合不同请求内容返回冲突；相同已完成请求可返回持久化结果，但不会再次生成分录。幂等记录和账本提交处于同一事务。

最终执行先检查当前策略及授权快照，随后才处理缓存，因此旧工作流不能借助缓存绕过策略失效。并发请求由 BEGIN IMMEDIATE 串行化写入，模拟分录还具备工作流唯一约束。重复执行只有一次模拟资金变化这一结论来自并发测试，而非前端按钮禁用。

实现与证据 backend/services/idempotency_service.py:15、26；backend/services/execution_gateway.py:126

13 Maker Checker 双人复核机制

Maker 是工作流所属的已认证主体。大额操作须在 Maker 确认和 MFA 后提交复核；Checker 必须具有服务端 supervisor 角色且 actor_id 与 Maker 不同。普通第二用户即使身份不同也不能审批。拒绝结果形成 REJECTED 终态，不能在原工作流重新放行。

审批记录持久化 checker_id、角色、approved_authorization_digest、approved_policy_version、approved_policy_fingerprint、approved_plan_hash 及 approved_at。旧接口提交 plan_hash，由后端绑定唯一完整快照。系统验证的是独立开发身份与角色；尚不能证明两个账号一定由两位自然人持有。

实现与证据 backend/services/execution_gateway.py:126；backend/services/api_v3.py:142；backend/services/auth_service.py:24

14 MFA 多因素认证

RED 流程先建立 Maker 确认摘要，再要求独立 MFA 操作。MFA 记录绑定 Maker 主体、同一 authorization_digest、验证时间和 300 秒有效期。最终执行及 Checker 审批再次验证记录，过期或摘要不匹配必须拒绝。

当前 MFA 是开发模式显式模拟，受 APP_ENV 和 DEMO_MFA_ENABLED 限制；它没有连接短信运营商、FIDO2、真实人脸或银行认证服务。界面中的验证交互仅演示状态机，不代表已经实施生产级多因素认证。

实现与证据 backend/services/execution_gateway.py:116、126；backend/main.py 的 MFA 路由

15 HITL 人工接管机制

满足资金约束的大额 RED 工作流在 Maker MFA 后进入 PENDING_HUMAN_REVIEW。发起人提交复核后成为 AWAITING_CHECKER，独立主管审批通过才能成为 APPROVED。在这两个等待状态中，后端禁止执行并记录拒绝，不能产生 execution_started。

人工接管不通过“点击后立即成功”的 UI 假状态实现。CheckerApprovalModal 使用独立主管开发口令建立另一会话，向后端提交审批。界面仅反映后端状态。生产中的组织授权、岗位互斥和人工风控工单集成仍待完成。

实现与证据 backend/services/execution_gateway.py:126；frontend/components/CheckerApprovalModal.jsx

16 Audit Trail 审计追踪

AuditService 在 SQLite 中追加事件，使用 UTC 时间、服务端主体、会话、工作流、动作、权限、状态、原因和耗时。敏感字段递归脱敏，不记录 MFA 密码或完整卡号。每条记录包含 previous_hash 和 event_hash，数据库触发器阻止普通 UPDATE 与 DELETE。

写入前验证现有链，金融执行和成功审计在同一事务中完成；写审计失败会阻止或回滚资金变化。REST 支持游标和过滤，SSE 推送 audit_event。哈希链能发现链内修改，但拥有数据库管理权限的人可移除触发器，因此不能宣称抗特权管理员篡改或外部公证。

实现与证据 backend/services/audit_service.py:12、49、61、91；backend/services/api_v3.py:156、188

17 策略版本绑定及旧授权失效机制

PolicyService 将规范化交易内容、policy_version、代码与配置指纹、策略代次、plan_hash、沙箱标识和结果摘要、金额、对象、动作、幂等键摘要及 Maker 一起封装。服务端生成 authorization_digest 并持久化不可变 authorization_snapshots；客户端不能覆盖策略版本。

任一授权阶段及最终执行都会比较当前策略、原快照和授权记录。版本或指纹变化使未完成工作流进入 POLICY_STALE，清除旧 Maker/MFA/Checker 授权并使沙箱授权失效。即使版本回滚，策略代次也阻止旧审批复活；必须创建新工作流重新授权。策略激活与资金执行使用同一数据库写锁。

实现与证据 backend/services/policy_service.py:21、39、84、107、120；backend/services/execution_gateway.py:104

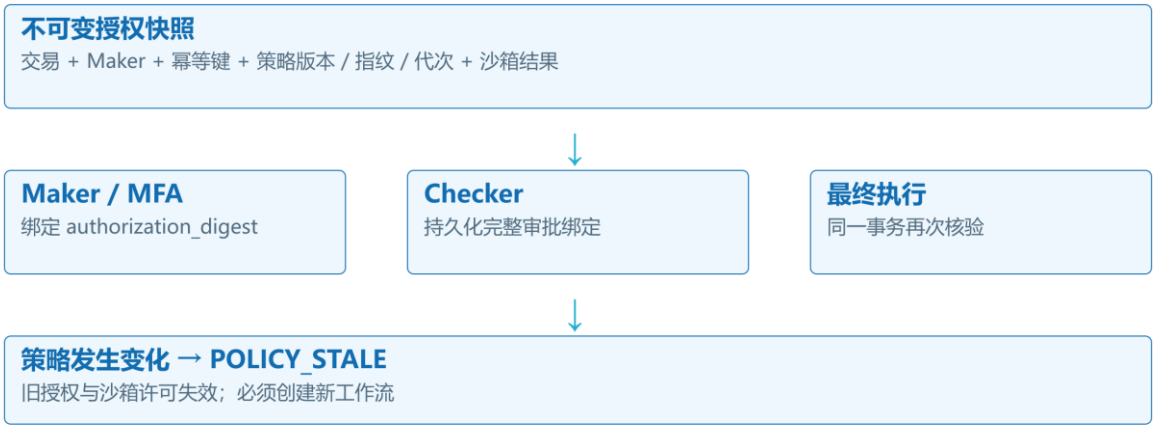


图 6 不可变授权快照与策略失效机制。

18 MCP 工具调用体系

MCPConnectorGateway 提供受控工具目录及 MockBankConnector 只读数据。mcp_server 暴露项目已有工具能力，保留账户、账单、卡片、产品及权限判断等接口。工具调用同样受身份委托与动作权限约束，无法获取任意 SQL 写入能力。

真实转账动作 transfer.execute 始终阻断。其他写操作形状的工具返回安全判定或模拟描述，不直接产生账本副作用；所有资金变动只能通过独立 ExecutionGateway。MCP 专项测试检查绕过路径，不把“工具存在”描述为真实银行接口已接入。

实现与证据 backend/agent_runtime.py:414、452；backend/mcp_server.py；backend/tests/test_banking_v3.py

19 SSE 实时任务通信

TaskEventBus 将主控启动、领域选择、节点校验与卡片生成进度发送给前端，支持任务流展示。API 中审计流和工作流流从后端持久化事件提供新增记录与游标，按已认证主体过滤；其他用户不能读取不属于自己的任务流。

任务 EventBus 主要保存在进程内，不能宣称重启后所有瞬时任务进度完整续传。持久化审计和决策文档可以在重启后重新查询与回放。客户端需处理断线、会话过期、游标和重复事件，界面中的实时连接不等于交易授权。

实现与证据 backend/agent_runtime.py:297、304；backend/services/api_v3.py:61、181、193

20 数据结构与接口设计

主要数据表包括 workflows、sessions、simulated_accounts、simulated_entries、idempotency_records、audit_events、decision_documents、financial_scenarios、active_policy 和 authorization_snapshots。工作流 JSON 保存状态及绑定信息；会话仅存令牌哈希和有效期；金额以分保存。

交易入口 POST /api/transactions 创建独立工作流。随后使用 /maker-confirm、MFA 路由、/submit-review、/human-review 和 /execute。接口采用 StrictModel 拒绝额外字段，角色从认证中间件取得。409 表示状态或摘要冲突，403 表示授权不足，503 表示审计或生产身份不可用；完整接口以随包文档和运行时 /openapi.json 为准。

实现与证据 backend/services/database.py；backend/services/api_v3.py:25、119；backend/main.py

21 部署与运行说明

已验证环境为 Windows、Python 3.13.12、Node.js 26.13.0，前端 Next.js 15.5.28 与 React 19.1.0。后端依赖在 requirements 系列文件固定；安装使用独立虚拟环境和 npm ci，避免无检查升级。源码包提供安全默认的 .env.example、初始化命令、启动与测试脚本。

默认 APP_ENV=production 将拒绝未配置生产身份的请求。仅在本地演示时显式切换 development，设置不同的随机开发口令并保持 BANKING_DB_PATH 指向新的演示数据库。initialize_database.py 创建结构，策略表在首次授权时初始化。外部模型 Key 可为空，本地规则回退可运行。

实现与证据 backend/requirements*.txt；frontend/package.json；backend/initialize_database.py；INSTALL.md

22 测试验证结果

最终冻结源码的完整批次共 90 项，通过 90、失败 0、错误 0、跳过 0。文件分布为 banking_v3 41 项、security_acceptance 2 项、intelligence 12 项、final_security_audit 15 项、policy_binding_v31 20 项。43、55、70 项历史范围均独立重跑通过，但属于相互包含的集合。

新增策略测试使用独立临时 SQLite 数据库及真实 HTTP 服务进程，覆盖不同阶段策略变化、同版本指纹变化、旧审批、并发、审计失败和沙箱不一致。核心负向场景返回拒绝并进入 POLICY_STALE，余额与分录变化为零，execution_started/completed 为零，链验证通过。前端构建及 pip check 已通过；本轮制作不冒充重新执行了安全测试。

实现与证据 backend/test-results/v31-security/results.json；full/v31-acceptance.xml；frontend-build-output.txt

23 应用场景与商业价值

六大场景将分散的金融动作组织为可理解的任务：资金规划可解释预留；转账风险提示可暴露陌生对象与大额风险；账单分析提供分类洞察；理财适配要求独立授权；卡片保护及订阅管理体现可控操作。

潜在价值是减少操作衔接中的信息遗漏、提高授权依据的可追溯性，并为智能体应用提供可测试的安全执行框架。尚无真实用户实验、成本收益测算、转化率或机构采购证据，因此不量化效率提升或收入规模。商业验证应先从隔离环境概念验证开始。

实现与证据 frontend/pages/index.js 的六大场景；backend/agent_runtime.py:500、558

24 技术创新与未来发展

本项目的工程特色是把语言规划、动态解释界面和强制执行网关连接起来，并将授权对象从单一布尔确认提升为包含交易、策略与沙箱的不可变快照。策略变化使旧授权不可恢复，幂等、账本与审计共享事务，避免前端确认与后端真实授权脱节。

未来计划包括机构 IdP 与强认证接入、外部可信审计锚点、生产连接器审查、权限岗位分离、多进程事件总线及可观测性。每一步需独立威胁建模与验证；这些属于路线图，不是已交付能力，也不宣称专利、首创或优于所有同类系统。

实现与证据 backend/services/policy_service.py；backend/services/execution_gateway.py；本文件第 25 节

25 当前系统边界及局限性

系统仅使用模拟账户、模拟产品和独立账本。开发身份不等于银行实名身份；模拟 MFA 不等于真实多因素认证；两个不同开发账号不证明两位自然人隔离。规则与数字孪生结果不是投资建议、信用评级或经过校准的金融预测。

SQLite 适合当前单机演示，未完成银行级容量、灾备、外部渗透测试和合规认证。审计链无外部锚定，生产 IdP 未配置，真实银行接口完全未接入。验收证明的是明确测试范围内的安全属性，不是对未知漏洞、实际部署或未来代码的无条件保证。

实现与证据 FINAL_SECURITY_ACCEPTANCE_V3_1.md 安全边界；backend/services/auth_service.py:47

附录 A 验收数量与复核入口

批次	总数	通过	失败/错误/跳过
历史基础安全	43	43	0 / 0 / 0
历史含智能分析	55	55	0 / 0 / 0
原最终安全审计	70	70	0 / 0 / 0
最新完整独立集合	90	90	0 / 0 / 0

集合关系为 43 ⊂ 55 ⊂ 70 ⊂ 90。原 70 项与新增策略测试 20 项互斥，组成完整 90 项。其他批次不可重复累加。

测试文件	用例数
test_banking_v3.py	41
test_security_acceptance.py	2
test_intelligence.py	12
test_final_security_audit.py	15
test_policy_binding_v31.py	20

实现与证据 backend/test-results/v31-security/results.json 与 full/v31-acceptance.xml；源码交付包 evidence/ 提供冻结证据副本。

从源码包 backend 目录执行：python -m pytest tests -q --tb=short --junitxml=test-results/recheck.xml。安全测试使用临时数据库与独立端口。测试可能写入自身 test-results 目录，复核时使用解压副本，不覆盖原验收证据。

图 2—6 的可编辑 SVG 随文档提供；PPT 中对应结构为原生可编辑形状。截图为真实界面裁切，没有重绘或替换交易状态。